When to use Flink SQL `LATERAL VIEW` vs. a `ProcessTableFunction` (PTF)

- 1.0 The guideline
- 2.0 A practical version of the rule
- 3.0 What "minimizing the complexity of the UDF" should look like in practice
- 4.0 The mental model

1.0 The guideline

Use plain FlinkSQL (including `LATERAL`, `UNNEST`, `CROSS JOIN`, and built-in table functions) for everything it can already express. Reach for a `ProcessTableFunction` only when you need something Flink SQL fundamentally cannot do — and when you do, make the PTF's job *only* that thing.

The reasons this is the right default:

1. **The optimizer can see through Flink SQL; it cannot see into a UDF.** Predicate pushdown, projection pushdown, join reordering, filter fusion, watermark alignment — all of that stops at the boundary of a Java function. Every column you read inside `eval()` is a column the planner must materialize for you whether you end up using it or not.
2. **Flink SQL is portable and declarative.** A `LATERAL VIEW` query runs unchanged on a new Flink version, on a managed service, on Confluent Cloud, on a unit test with a mini-cluster. A PTF is Java you have to compile, package, register, and version.
3. **State and timers are the actual hard part.** That's where bugs live (TTL, key cardinality, checkpointing cost, late events, timer storms). Concentrating that complexity in one small, well-named, well-tested PTF is exactly the kind of isolation you want.
4. **Schema evolution is easier in Flink SQL.** Adding a column to an `UNNEST` chain is a one-line edit. Adding a column to a PTF means touching `@DataTypeHint`, the `Row.of(...)` call, the tests, and rebuilding the JAR.

2.0 A practical version of the rule

Think of it as a decision ladder — stop at the first rung that works:

1. **Pure Flink SQL** (projections, filters, joins, window TVFs, `MATCH_RECOGNIZE`).
2. **Flink SQL + built-in table functions** (`UNNEST`, `LATERAL`, `JSON_TABLE`, `STRING_SPLIT`, …).
3. **Scalar / table / aggregate UDF** — when you need a small piece of custom logic but no cross-row memory.
4. **Row-semantic PTF** — when per-row logic is too gnarly for a scalar UDF *and* you want it to look like a table operator in Flink SQL (this is what `OrderLineExpander` demonstrates; honestly, in production you'd usually do this one in Flink SQL).
5. **Set-semantic PTF with state/timers** — when, and only when, you need keyed state, event-time timers, custom windowing, sessionization with bespoke gap rules, dedup-with-memory, "emit on Nth event," etc. *This is the rung where PTFs justify their existence.*

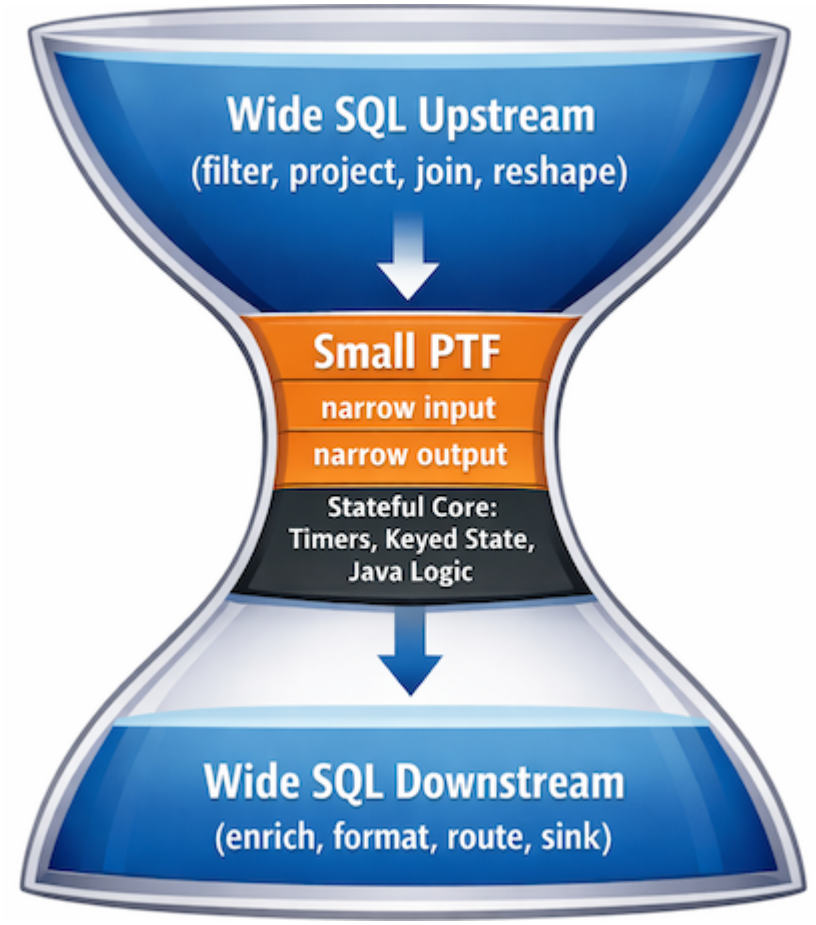
3.0 What "minimizing the complexity of the UDF" should look like in practice

When you do write a stateful PTF, keep these disciplines:

- **Push everything you can upstream into Flink SQL.** Filter, project, and pre-join in Flink SQL *before* the data hits the PTF. The PTF should receive the narrowest possible table — only the columns and rows it actually needs to make state decisions. This shrinks state size, checkpoint size, and CPU.
- **Push everything you can downstream into Flink SQL.** The PTF should emit the *minimum* facts needed; let Flink SQL do the formatting, enrichment joins, and final shaping. Don't have the PTF do a lookup join that a Flink SQL `JOIN` could do after the fact.
- **One PTF, one responsibility.** Don't build a "god PTF" that sessionizes *and* dedupes *and* enriches *and* formats. Chain small ones, or split the work between a small PTF and surrounding Flink SQL.
- **Keep `eval()` boring.** If a method on the PTF is doing string parsing or arithmetic gymnastics, that logic almost always belongs in a Flink SQL expression or a tiny scalar UDF, not inside the stateful operator.
- **Make the state explicit and named.** Future-you (and the checkpoint-restore story) will thank you.
- **Test the PTF in isolation** with a `TableEnvironment` mini-job. Because it has a narrow contract (table in, table out), it's straightforward to test if you've kept its responsibility small.

4.0 The mental model

A good PTF in a Flink SQL pipeline is shaped like an **hourglass**:



The PTF is the pinch point. Keep everything declarative in the wide top and bottom. The Java in the middle stays small and focused, used only where **process time/event time, keyed state, and control over when**

to emit, what to emit, and why go beyond Flink SQL.

One-line summary: *Flink SQL for shape, PTF for memory.*